Arab Open University- Branch Name


Faculty of Computer Studies

Information Technology and Computing Department

# ***<SharkTalent>***

Name and ID: Ali Tarek Mohamed, 210661

TM471: Final Year Project, May 2025

Supervisor: Dr. Manief

# Abstract

The tech freelance world has problems like too much spam in applications, reviews you can't trust, and talks happening all over the place. It's hard for clients to find good freelancers, and sometimes projects aren't finished, or payments cause fights because things aren't set up well.

SharkTalent wants to fix this with a clear scoring system to suggest the right freelancers. Freelancers can only apply to three projects, which cuts down on spam. Clients get to chat with freelancers in one place, plan payments step-by-step, and see how things are going easily.

This project will build a complete website for tech freelancers and clients. It will have different logins for each type of user and secure ways to sign up, manage profiles, bid on jobs, and leave ratings. SharkTalent focuses on being easy to use, keeping everyone responsible, and offering a safe space to learn.

# Acknowledgement

I really appreciate everyone who helped me with this project.

First, thanks a lot to my supervisor, Dr. Manief. Their advice and support were super helpful at every step. They really guided me, and their knowledge made this project much better.

I also want to thank the teachers in IT Department. What I've learned from them about software development and systems analysis has provided the foundation for this graduation project I'll be working on.

A big thanks to my friends and family for being patient, supportive, and keeping me going. Their belief in me helped me keep trying.

Lastly, I'm thankful for all the online stuff, research papers, and tools that’s helping me initiating a design and plan the SharkTalent freelance platform.

# Table of Contents

# CHAPTER 1 – INTRODUCTION

## 1.1 Overview

SharkTalent is a prototype freelance platform designed to tackle major flaws in existing systems like spammy proposals, unreliable ratings, and poor communication. Focused on the tech industry, it introduces a rule-based structure with verified skill quizzes, badge systems, and simulated milestone payments. The platform aims to promote fairness, transparency, and simplicity, avoiding real transactions and AI-based sorting in favor of clean design and structured workflows.

Targeted at tech clients, freelancers, and educational users, SharkTalent supports core features like proposal caps, in-app messaging, dashboard tracking, and built-in feedback. This project acts not only as a functional solution but also as a testbed for academic or safe-development use, offering a clearer, more reliable approach to freelance work online.

## 1.2 Main Goal

The primary goal of this project is to design and develop SharkTalent, a web-based freelance platform tailored for the tech industry. It aims to address key issues in existing platforms such as spam-filled proposals, unreliable feedback, and scattered communication by introducing a transparent, rule-based system that emphasizes quality, fairness, and usability.

Instead of relying on traditional rating algorithms or AI filtering, SharkTalent will implement:

- A cap of three proposals per freelancer per project to reduce noise
- Verified skill assessments through quizzes and badge awards
- Structured post-project feedback and review mechanisms
- A simulated milestone-based payment system for safe trial use
- A unified dashboard for real-time communication and progress tracking

By focusing on structured user flows and transparent scoring, this platform also serves as a prototype for educational and experimental environments, offering a safe space to test freelance processes without involving legal or financial risk.

## 1.3 Objectives

- Analyze existing freelance platforms to identify key limitations and user experience challenges.
- Design a clean, intuitive user interface with a focus on minimalism and ease of use.
- Develop an integrated dashboard for clients and freelancers to manage tasks, proposals, and ongoing projects.
- Implement secure authentication, including user registration, login, and profile customization features.
- Introduce a skill verification system using quizzes and digital badges linked to specific project categories.
- Limit freelancers to a maximum of three proposals per project to reduce proposal spam and promote thoughtful applications.
- Simulate milestone-based payment flows to mirror real-world transactions without involving actual funds.
- Build an internal messaging and progress tracking system to centralize project communication and updates.

## 1.4 Target Audience

SharkTalent is designed to serve a focused group of users within the freelance and academic ecosystem, including:

- **Tech industry clients** seeking reliable, skilled freelancers for specialized project work.

- **Freelancers** such as developers, designers, and testers looking for fair, merit-based opportunities.
- **Startups and small businesses** in need of streamlined project execution without the overhead of complex platforms.
- **Educators, students, and researchers** interested in experimenting with freelance marketplace models in a risk-free, simulated environment.

The platform balances practical application with academic flexibility, making it suitable for both real-world use and educational exploration.

## 1.5 Project Scope

**In Scope**:

- Prototype development for a freelance web platform
- UI for posting, searching, and applying to tech-based projects
- Core functionalities: login, dashboard, badge system, simulated payment, feedback
- Data validation, testing, and user flow simulations

**Out of Scope**:

- Real monetary transactions or third-party payment gateway integrations
- Deployment at a commercial level
- Mobile app development

## 1.6 Problem Definition

Despite their popularity, existing freelance platforms often fail to ensure fair project matching and clear communication. Clients are overwhelmed with spammed, repetitive proposals, while freelancers are judged based on inflated or manipulated ratings. Communication is typically scattered across external channels, and the lack

of structured feedback mechanisms leads to untracked disputes or abandoned projects. There's also no safe environment to simulate the dynamics of freelance transactions for academic or developmental testing.

## 1.7 Suggested Solution

The proposed solution, **SharkTalent**, is a web-based freelance platform that addresses these challenges through:

- Rule-based limitations (e.g., 3 proposals per freelancer per job)
- Verified quizzes and skill badges tied to specific job categories
- A post-project review system built into the platform
- A simulated milestone payment feature (without real transactions)
- Unified dashboards for tracking proposals, tasks, messages, and timelines

The platform will not rely on keyword matching or AI ranking algorithms; instead, it emphasizes fairness, skill validation, and clean user interface design to ensure effective and transparent interactions.

## 1.8 Conclusion and Perspective

SharkTalent is envisioned as more than just another freelance platform it is a **structured and testable prototype** aimed at redefining freelance project interactions through rule-driven validation and interaction control. The focus is not only on functional effectiveness but also on academic and design clarity, making the system applicable in both real-world and learning environments. By balancing system constraints with user freedom, this platform may contribute to broader discussions on ethical, efficient, and transparent freelance ecosystems in the digital age.

## 1.9 Gnatt's Chart

| ID | Task Name | Start | End | Durat |
|---|---|---|---|---|
| 1 | Project Report | 2025-03-03 | 2025-07-25 | 105 d |
| 4 | Introduction | 2025-03-03 | 2025-03-24 | 16 da |
| 27 | Aims & Objectives | 2025-03-03 | 2025-03-10 | 6 day |
| 28 | Deliverables & Feasibility Study | 2025-03-10 | 2025-03-17 | 6 day |
| 26 | Project Background & Problem Statement | 2025-03-17 | 2025-03-24 | 6 day |
| 19 | Literature Review | 2025-03-24 | 2025-04-14 | 16 da |
| 31 | Existing Softwares | 2025-03-24 | 2025-03-31 | 6 day |
| 29 | Introduction & Description | 2025-03-31 | 2025-04-07 | 6 day |
| 30 | Types & Purpose of Literature Review | 2025-04-07 | 2025-04-14 | 6 day |
| 21 | Requirement Analysis | 2025-04-14 | 2025-05-05 | 16 da |
| 37 | Data Collection | 2025-04-14 | 2025-04-18 | 5 day |
| 39 | Interviews | 2025-04-18 | 2025-04-25 | 6 day |
| 38 | Questionaires | 2025-04-25 | 2025-05-01 | 5 day |
| 40 | System Requirements | 2025-04-28 | 2025-05-05 | 6 day |
| 41 | Functional Requirements | 2025-04-28 | 2025-05-05 | 6 day |
| 42 | Non-Functional Requirements | 2025-04-28 | 2025-05-05 | 6 day |
| 20 | Research | 2025-05-05 | 2025-06-13 | 30 da |
| 35 | Choice of Model | 2025-05-05 | 2025-05-16 | 10 da |
| 36 | Justification for Model | 2025-05-16 | 2025-05-23 | 6 day |
| 33 | Research Approach | 2025-05-23 | 2025-05-30 | 6 day |
| 32 | Research Methods | 2025-05-30 | 2025-06-06 | 6 day |
| 34 | Software Development Life-Cycle | 2025-06-06 | 2025-06-13 | 6 day |
| 22 | System Design | 2025-06-13 | 2025-07-25 | 31 da |
| 43 | Architecture Design | 2025-06-13 | 2025-06-20 | 6 day |
| 45 | DFD Diagrams | 2025-06-20 | 2025-06-27 | 6 day |
| 48 | ER Diagram | 2025-06-27 | 2025-07-04 | 6 day |
| 47 | System Design Concept | 2025-07-04 | 2025-07-11 | 6 day |
| 49 | UML Diagram | 2025-07-11 | 2025-07-18 | 6 day |
| 44 | Use Case Diagram | 2025-07-11 | 2025-07-18 | 6 day |
| 46 | Wireframe Design | 2025-07-18 | 2025-07-25 | 6 day |
| 12 | Software Development | 2025-07-28 | 2025-12-30 | 112 d |
| 3 | Testing | 2025-07-28 | 2025-12-30 | 112 d |
| 8 | Features to Test | 2025-07-28 | 2025-10-30 | 69 da |
| 9 | Testing Environment | 2025-10-30 | 2025-12-30 | 44 da |



Figure 1: Gnatt's Chart for SharkTalent's development

# CHAPTER 2 – LITERATURE REVIEW

## 2.1 Overview

This chapter reviews three major freelance platforms Upwork, Fiverr, and Toptal to understand their features, strengths, and shortcomings. By comparing their models for proposals, payments, and user verification, we identify key issues that SharkTalent aims to solve. The chapter concludes with a summary table highlighting how SharkTalent improves upon existing systems through structure, transparency, and educational design.

## 2.2 Similar Project

### 2.2.1 - Upwork


Figure 2: Upwork

**Overview:**

Upwork is one of the largest freelance marketplaces, with millions of freelancers and clients participating globally. The platform supports a wide variety of job categories including programming, UI/UX design, data entry, and writing. Upwork operates primarily on a **bidding-based model**, where clients post jobs and freelancers submit proposals. Employers can then review profiles, ratings, work histories, and hourly rates to make hiring decisions.

**Key Features:**

- Escrow payment system with protection for both clients and freelancers.
- Proposal bidding with optional cover letters, attachments, and pricing.
- Client and freelancer rating systems, visible to both parties.
- Time tracking software for hourly projects.
- Freelancer categorization into levels (e.g., Rising Talent, Top Rated).

**Advantages:**

- Large user base with a vast talent pool.
- Payment protection via escrow.
- Flexibility in hiring hourly or fixed-price contracts.
- Integrated collaboration tools and milestone payments.

**Disadvantages:**

- High competition leads to proposal spamming and under-pricing.
- Platform takes a 20% fee from freelancers on initial earnings.
- Reviews and ratings can be manipulated, affecting trust.
- **Client overload** with too many similar applications, making shortlisting difficult.

### 2.2.2 - Fiverr


Figure 3: fiverr

## Overview:

Fiverr operates on a **gig-based model**, where freelancers list their services as predefined packages rather than applying to jobs. Clients browse and select from a catalogue of offerings based on ratings, price, and delivery time. Fiverr initially launched with the concept of "$5 gigs" but has evolved to support tiered pricing and more complex services.

## Key Features:

- Gig listing model instead of job proposals.
- Tiered service packages (Basic, Standard, Premium).
- Seller levels determined by sales performance and reviews.
- Quick delivery options for clients needing fast results.
- Basic dashboard with messaging and order tracking.

## Advantages:

- Easy onboarding for new freelancers.
- Simplified purchasing process for clients.
- Offers a wide range of creative and digital services.
- Enables passive income by maintaining active gigs.

## Disadvantages:

- High **service fee** (20% cut on every order).
- Limited customizability for clients with complex needs.
- No real-time vetting or testing of freelancer skillsets.
- Service quality can vary significantly due to low entry barriers

### 2.2.3 - Toptal


Figure 4: Toptal

## Overview:

Toptal differentiates itself through an **exclusive model**, marketing access to the “top 3%” of freelancers in tech and finance. It uses a **multi-phase screening process** including coding tests, portfolio reviews, and behavioural interviews. Once accepted, freelancers are matched to projects through internal recruiters and project managers.

## Key Features:

- Multi-stage vetting process with high rejection rates.
- Personalized freelancer-client matching.
- Trial period to assess freelancer performance risk-free.
- High-touch support model with project managers.
- Focused on high-budget, long-term contracts.

## Advantages:

- Ensures high-quality freelancer pool.
- Ideal for enterprise-grade projects.
- Offers dedicated client support and handholding.
- Trial periods reduce hiring risk.

## Disadvantages:

- Very high entry barrier for freelancers.
- Expensive for clients and inaccessible to small businesses.
- Time-consuming onboarding and screening process.
- Not suitable for quick, low-cost, or short-term projects.

## 2.3 Comparison Summary

| Criteria | Upwork | Fiverr | Toptal | SharkTalent |
|---|---|---|---|---|
| **Spam Control** | ✗ None | ✗ None | ✓ Curated | ✓ Proposal limits (max 3) |
| **Skill Validation** | ✗ Minimal | ✗ None | ✓ High | ✓ Badge system with quizzes |
| **Payment Safety** | ✓ Escrow (real) | ✗ None | ✓ Contracts | ✓ Simulated payments (safe for trials) |
| **Transparency** | ⚠ Variable | ✗ Low | ✓ High | ✓ Central dashboard with full tracking |
| **Educational Use** | ✗ Not suitable | ✗ Not suitable | ✗ Not open-source/testable | ✓ Designed for academic/test environments |
| **Cost Barrier** | ⚠ Medium fees | ✗ High fees (20%) | ✗ Very High | ✓ No real financial barriers |

**Table 2:** comparison between similar systems & shark talent

## 2.4 Summary of Literature Findings

The review of Upwork, Fiverr, and Toptal reveals several recurring issues across current freelance platforms. While each platform offers valuable features such as escrow payments, large talent pools, or advanced vetting they often fall short in ensuring proposal quality, verifying freelancer skills, and maintaining transparent, centralized communication.

Upwork struggles with oversaturation and manipulated reviews. Fiverr simplifies transactions but lacks skill testing and customization. Toptal offers high quality but at the cost of accessibility and speed. These insights directly inform SharkTalent's design, which aims to balance structure with flexibility through features like proposal limits, skill-based quizzes, badge validation, and simulated payments.

Overall, the literature highlights a clear need for a freelance platform that is fair, skill-driven, and educationally adaptable gaps that SharkTalent is specifically built to fill.

# CHAPTER 3 – REQUIREMENTS AND ANALYSIS

## 3.1 Overview

This chapter outlines the technical foundation of the SharkTalent platform by defining its requirements and development tools. It begins with the core functional features needed to support clients and freelancers such as registration, project posting, proposal submission, skill quizzes, and communication tools. Non-functional requirements like usability, performance, security, and reliability are also described to ensure the platform operates smoothly and meets user expectations.

The chapter then lists the software and hardware needed to develop and run the system, followed by a selection of modelling tools used for planning and visualization. Together, these requirements guide the design and implementation of SharkTalent as a functional and testable freelance platform prototype.

## 3.2 Functional Requirements

These are the features that SharkTalent will offer to both clients and freelancers:

- **User Registration and Login**: Users can sign up, log in, and choose whether they are a client or freelancer.
- **Profile Management**: Users can edit their profile, upload a picture, add skills, and show a portfolio.
- **Dashboard Access**: Each user has a dashboard to manage their activity—projects for clients, and proposals for freelancers.
- **Project Posting (Client)**: Clients can post new projects, add descriptions, set budgets, and choose required skills.
- **Proposal Submission (Freelancer)**: Freelancers can apply to projects but only send 3 proposals at a time.
- **Skill Quiz and Badges**: Freelancers can take short quizzes to earn badges that show verified skills.
- **In-App Messaging**: Clients and freelancers can message each other inside the platform.
- **Milestone Tracking**: Clients can break projects into steps (milestones), and both sides can track progress.
- **Post-Project Rating**: After completing a project, clients can leave a review and rating for the freelancer.

## 3.3 Non-Functional Requirements

These are qualities the system should have to work well for users:

- **Simple and Clean Design**: Easy for both clients and freelancers to use.
- **Fast and Responsive**: Works smoothly on computers, tablets, and phones.
- **Secure**: Keeps user data private and prevents unauthorized access.
- **Stable**: Doesn’t crash or lose data, even with many users online.
- **Testable**: Allows safe testing of features without real money involved.
- **Lightweight**: Works on average internet connections and devices.
- **Maintainable**: Easy to fix or update if something breaks later on.

## 3.4 Software Requirements

Below is a list of software components and frameworks needed for development and testing:

- **Frontend:**
  - React.js
  - JavaScript
- **Backend:**
  - Node.js with Express.js (API & Logic)
  - SQLalchemy

## 3.5 Hardware Requirements

Minimum and recommended configurations to deploy and run the **SharkTalent** platform (for local or institutional testing):

| Component | Minimum | Recommended |
|---|---|---|
| CPU | Intel i5 / AMD Ryzen 5 | Intel i7 or equivalent |
| RAM | 8 GB | 16 GB |
| Storage | 256 GB SSD | 512 GB SSD |
| Operating System | Windows 10 / Ubuntu 20.04+ | Windows 11 / Ubuntu 22.04 LTS |
| Network | 5 Mbps connection | 50 Mbps+ for faster sync and testing |
| Browser | Chrome, Firefox | Latest version of Chrome (Dev Tools) |

Table 3: hardware component requirement

## 3.6 UML Diagrams Analysis

### 3.6.1 - Use Case diagram

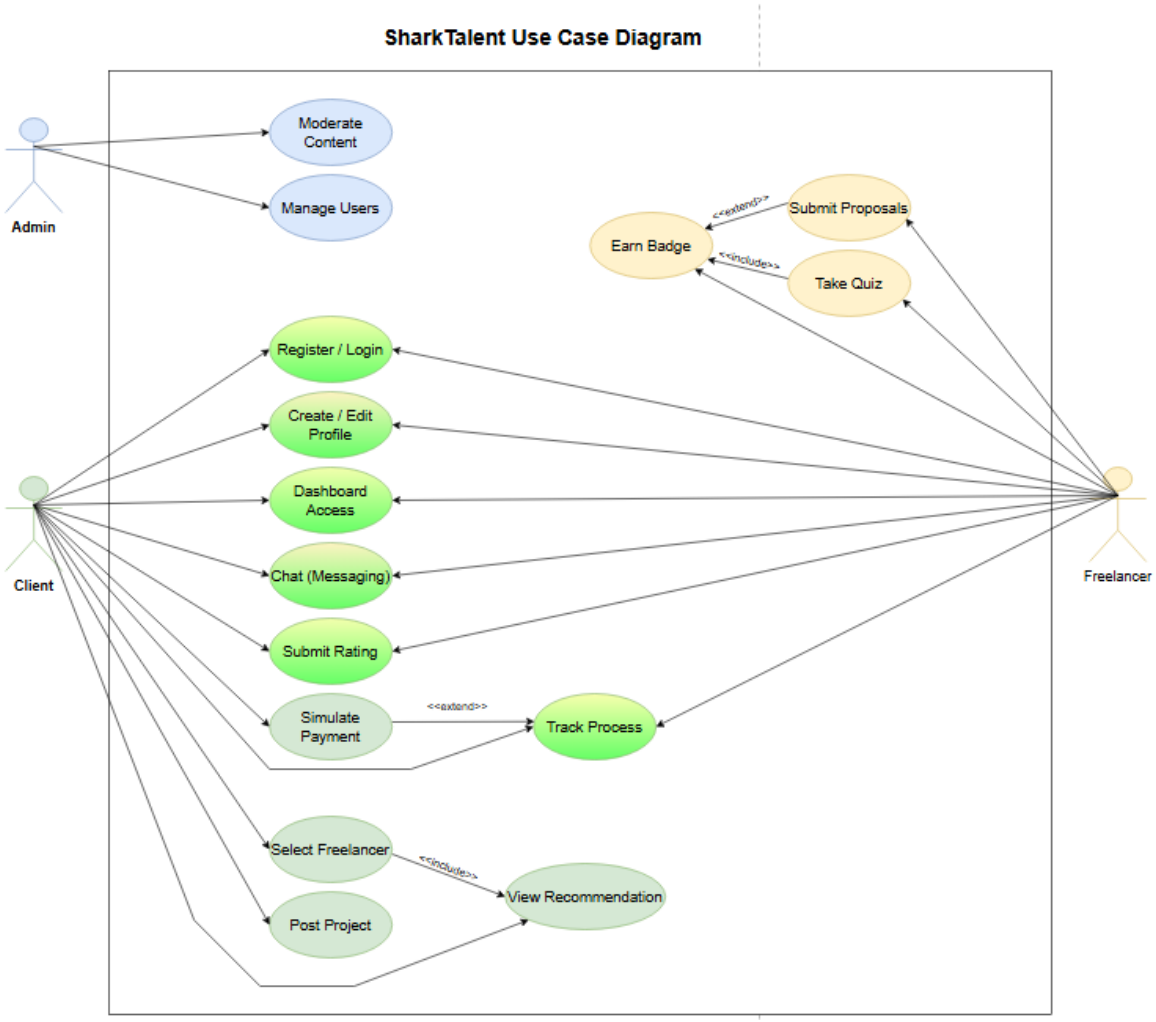


**Figure 5:** SharkTalent Use Case Diagram

**Admin**

The admin manages user accounts and makes sure the platform stays safe by removing inappropriate content.

**Client**

Clients can sign up, create a profile, post projects, and chat with freelancers. They select freelancers, track project progress, rate freelancers after work is done, and use a simulated payment system to approve milestones.

**Freelancer**

Freelancers register, build their profiles, and take quizzes to earn skill badges. They can view projects, send up to 3 proposals per job, chat with clients, track their tasks, and confirm milestones using the simulated payment flow.

### 3.6.2 - Use Case description

| | |
|---|---|
| **Use Case** | **Manage Users** |
| **Actor** | **Admin** |
| **Precondition** | Admin is logged in |
| **Postcondition** | Users are created, updated, or removed |
| **Main Flow** | 1. Admin logs in<br>2. Accesses "User Management"<br>3. Views list of users<br>4. Selects user to edit or delete<br>5. Makes changes<br>6. System confirms changes |
| **Extensions** | - Step 3: If list fails to load → error displayed<br>- Step 5: Invalid changes (e.g. removing last admin) → validation error |

**Table 3:** Use Case Description for Admin

| | |
|---|---|
| **Use Case** | **Post Project** |
| **Actor** | **Client** |
| **Precondition** | Client is registered and logged in |
| **Postcondition** | New project appears on platform |
| **Main Flow** | 1. Client logs in<br>2. Opens dashboard<br>3. Clicks "Post Project"<br>4. Enters project details and milestones<br>5. Submits project<br>6. Project is listed |
| **Extensions** | - Step 4: If milestones not set → warning shown<br>- Step 5: If required fields missing → form validation prompts |

**Table 4:** Use Case Description for Client

| Use Case | Submit Proposal |
|---|---|
| Actor | Freelancer |
| Precondition | Freelancer is logged in and under proposal cap |
| Postcondition | Proposal is submitted to selected project |
| Main Flow | 1. Freelancer logs in<br>2. Browses projects<br>3. Clicks “Submit Proposal”<br>4. Enters pricing, timeline, and pitch<br>5. Submits proposal<br>6. Proposal is saved |
| Extensions | - Step 3: If cap (3 proposals) reached → block submission<br>- Step 5: If required fields are empty → prompt user |

**Table 5:** Use Case Description for Freelancer

## 3.7 Flowchart Diagram

### 3.7.1 Client Flowchart

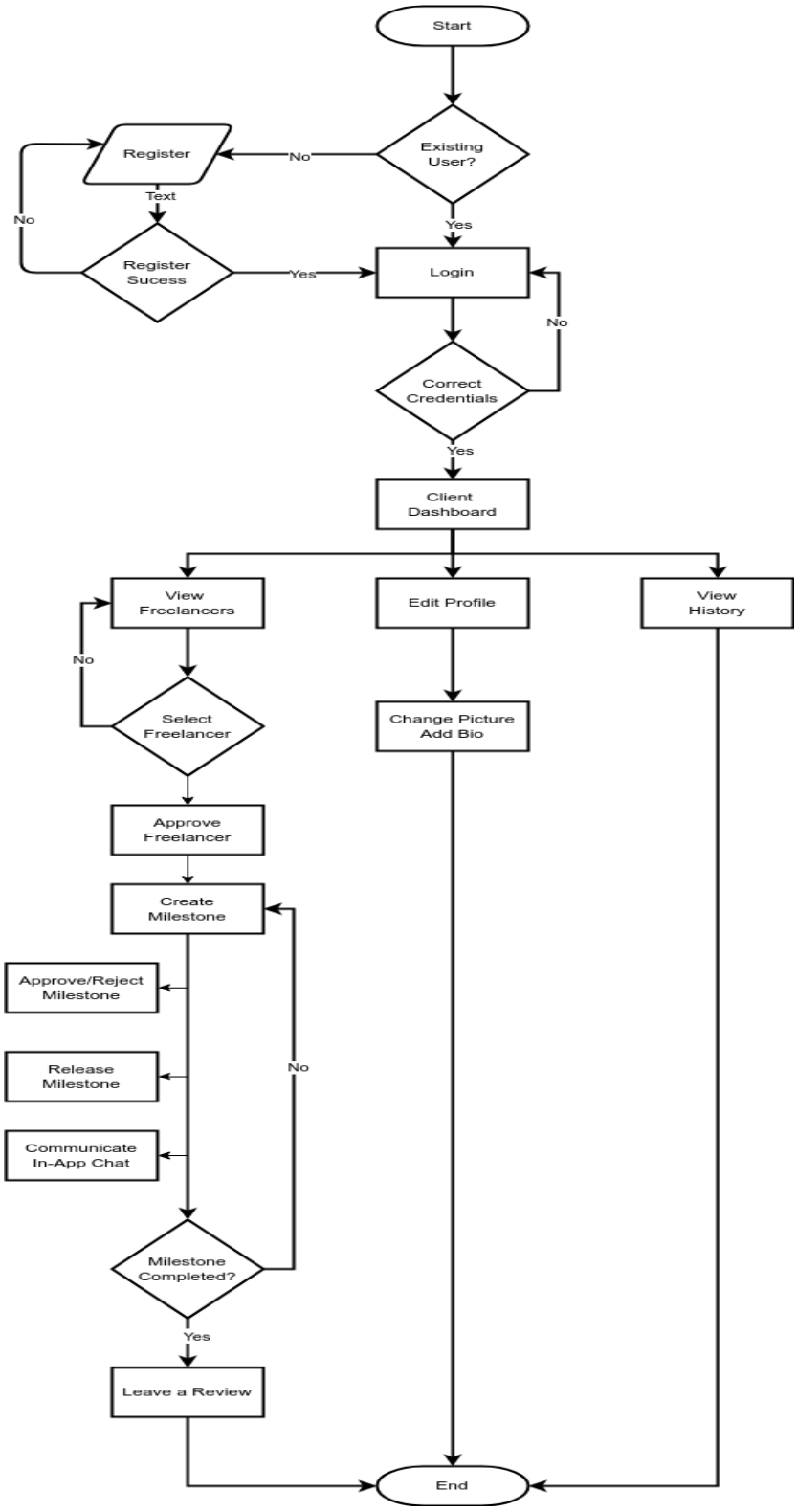


**Figure 6**: Client Flowchart Diagram

### 3.7.2 Freelancer Flowchart

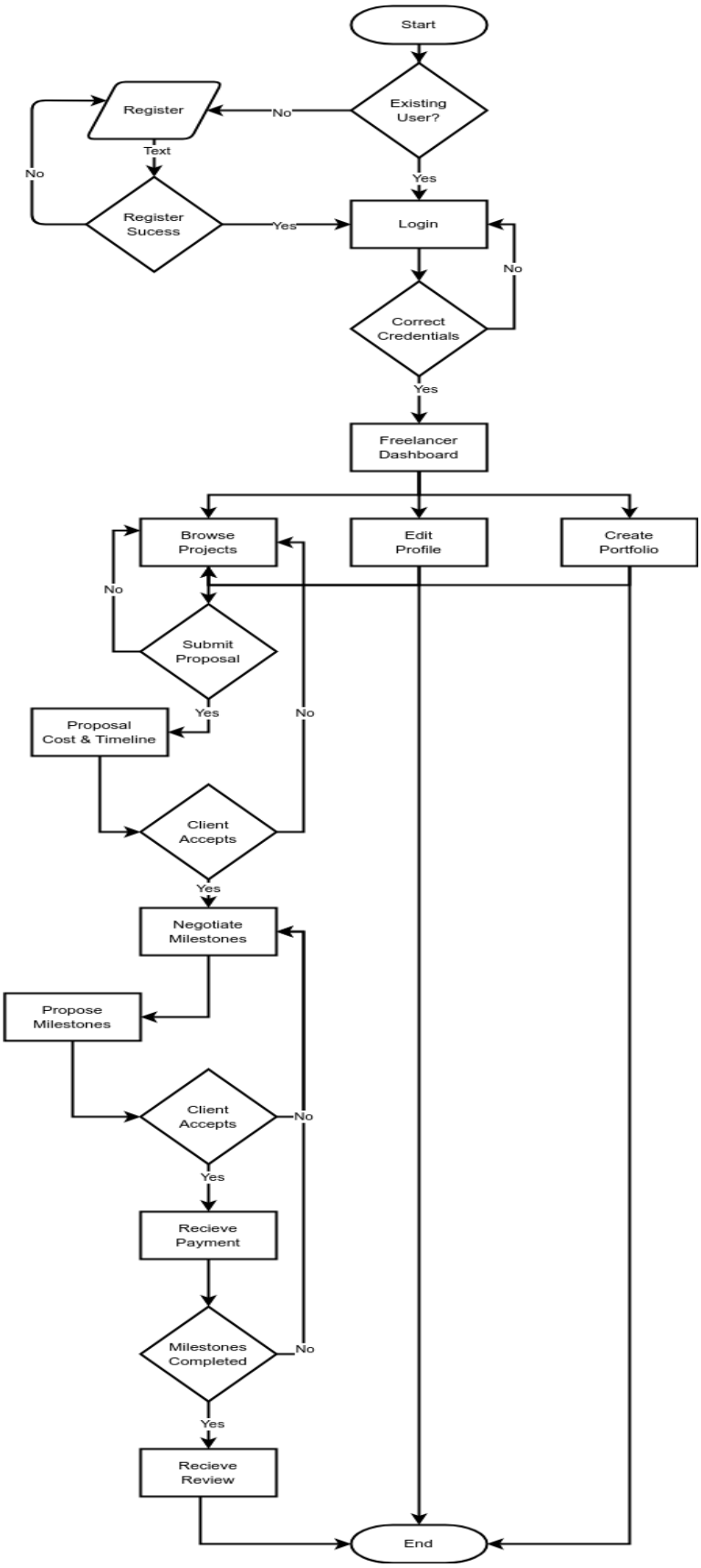


**Figure 7**: Freelancer Flowchart Diagram

# CHAPTER 4 – DESIGN, IMPLEMENTATION, AND TESTING

## 4.1 Overview

This chapter provides an in-depth account of the design methodology, architectural decisions, implementation process, and testing strategies adopted during the development of the SharkTalent platform. Building on the requirements established in Chapter 3, the system was engineered using a combination of structured design principles, modular decomposition, and iterative refinement. The decisions documented here respond directly to the challenges inherent in contemporary freelancing systems, including information overload, proposal spam, poor skill verification, and fragmented communication.

A key aim of this chapter is to justify the chosen design techniques and explain why they are appropriate relative to alternative approaches. Particular attention is given to areas where trade-offs were necessary such as the decision between monolithic and multi-tier architectures, synchronous vs. real-time communication, and the depth of skill-assessment mechanisms. This chapter also highlights the main implementation challenges encountered and the “coding traps” that emerged throughout development, providing insight into the reasoning behind critical technical decisions.

The chapter concludes by presenting the testing strategy, which follows a category-partition model to ensure structured evaluation, alongside functional testing, usability testing, and verification against expected system behaviours.

## 4.2 Design Techniques and Methodology

### 4.2.1 Architectural Design Approach

The architectural design for SharkTalent is based on an object-oriented domain model that emphasises the relationships between core business entities rather than adopting a strict three-layered architectural structure. This approach is appropriate because the platform is built around well-defined concepts such as users, projects, proposals, milestones, messages, ratings, and skill-verification components which naturally lend themselves to class-based modelling. Using a class diagram as the primary design tool provides clarity in understanding how the system behaves, how objects collaborate, and how data flows between modules.

The class diagram defines the main abstractions in the system, their attributes, and their associations. For example, the User class represents all actors in the platform and forms the central link between features such as project creation, proposal submission, quiz attempts, messaging, and ratings. The Project, Proposal, and Milestone classes model the lifecycle of a freelance job, enabling structured project breakdown and controlled workflow. Communication and feedback are represented through the Message and Rating classes, while platform credibility is reinforced through the Quiz, QuizQuestion, QuizResult, Badge, and UserBadge classes, which support skill verification and reputation building.

Class-based modelling provides several advantages for the SharkTalent system. It allows a clear separation of conceptual responsibilities, promotes modularity, and maps naturally to the SQLAlchemy ORM used in the implementation. By defining associations, multiplicities, compositions, and role-based interactions, the class diagram creates a blueprint that guides the implementation of database models, API endpoints, and business-logic functions. This technique also aligns with best practices in software engineering, where domain-driven modelling improves maintainability and ensures consistency between requirements, design, and implementation.

Overall, the class-diagram-centred design approach supports academic evaluation because it offers a structured and comprehensible representation of the system while remaining flexible enough to accommodate iterative development. It replaces the rigid layered model with a domain-focused view that better mirrors the functional and data-driven nature of the SharkTalent platform.

### 4.2.2 Comparison With Alternative Designs

During the design stage, several architectural approaches were evaluated before selecting a domain-model-driven, object-oriented architecture centred around the class diagram. This section explains why alternative designs were not chosen and why the adopted approach is the most appropriate for the SharkTalent platform.

**Monolithic Architecture**

A fully monolithic structure, where all logic, data handling, and workflows reside in a single undifferentiated codebase, was considered but rejected. Although monolithic systems are straightforward to implement initially, they become increasingly difficult to extend and maintain as complexity grows. For a platform like SharkTalent—which involves project workflows, proposals, messaging, quizzes, and ratings—a monolithic structure would quickly lead to tightly coupled components, reduced flexibility, and increased difficulty when modifying or adding features. This contradicts the project's requirement for clear modular separation and maintainable design.

**Layered (Three-Tier) Architecture**

A traditional three-tier model (presentation, application logic, data layer) was also assessed. While this approach is widely used in commercial environments, it abstracts the system at too high a level to effectively describe the internal relationships that govern SharkTalent's behaviour. The platform is driven by domain entities—such as users, projects, proposals, milestones, quizzes, and badges—that interact through complex business rules. Representing these interactions through a layered model fails to capture the structural dependencies and lifecycle constraints that are central to the system. A class-diagram-based architecture, by contrast, exposes these internal relationships explicitly and provides a clearer mapping to the SQLAlchemy ORM used during implementation.

**Microservices Architecture**

A microservices approach was rejected due to unnecessary complexity. Although microservices offer strong scalability and independent deployment, they require distributed configuration management, inter-service communication protocols, API gateways, and substantial DevOps overhead. These requirements are disproportionately heavy for an academic project and would not provide meaningful benefits given the controlled scope and moderate workload of SharkTalent. Moreover, microservices can obscure system-wide relationships, which are central to the educational value of the project.

## 4.3 System Design

### 4.3.1 Architectural Overview

The architectural design for SharkTalent is based on an object-oriented domain model that emphasises the relationships between core business entities rather than adopting a strict three-layered architectural structure. This approach is appropriate because the platform is built around well-defined concepts such as users, projects, proposals, milestones, messages, ratings, and skill-verification components which naturally lend themselves to class-based modelling. Using a class diagram as the primary design tool provides clarity in understanding how the system behaves, how objects collaborate, and how data flows between modules.

The class diagram defines the main abstractions in the system, their attributes, and their associations. For example, the User class represents all actors in the platform and forms the central link between features such as project creation, proposal submission, quiz attempts, messaging, and ratings. The Project, Proposal, and Milestone classes model the lifecycle of a freelance job, enabling structured project breakdown and controlled workflow. Communication and feedback are represented through the Message and Rating classes, while platform credibility is reinforced through the **Quiz**, **QuizQuestion**, **QuizResult**, **Badge**, and **UserBadge** classes, which support skill verification and reputation building.

Class-based modelling provides several advantages for the SharkTalent system. It allows a clear separation of conceptual responsibilities, promotes modularity, and maps naturally to the SQLAlchemy ORM used in the implementation. By defining associations, multiplicities, compositions, and role-based interactions, the class diagram creates a blueprint that guides the implementation of database models, API endpoints, and business-logic functions. This technique also aligns with best practices in software engineering, where domain-driven modelling improves maintainability and ensures consistency between requirements, design, and implementation.

Overall, the class-diagram-centred design approach supports academic evaluation because it offers a structured and comprehensible representation of the system while remaining flexible enough to accommodate iterative development. It replaces the rigid layered model with a domain-focused view that better mirrors the functional and data-driven nature of the SharkTalent platform.

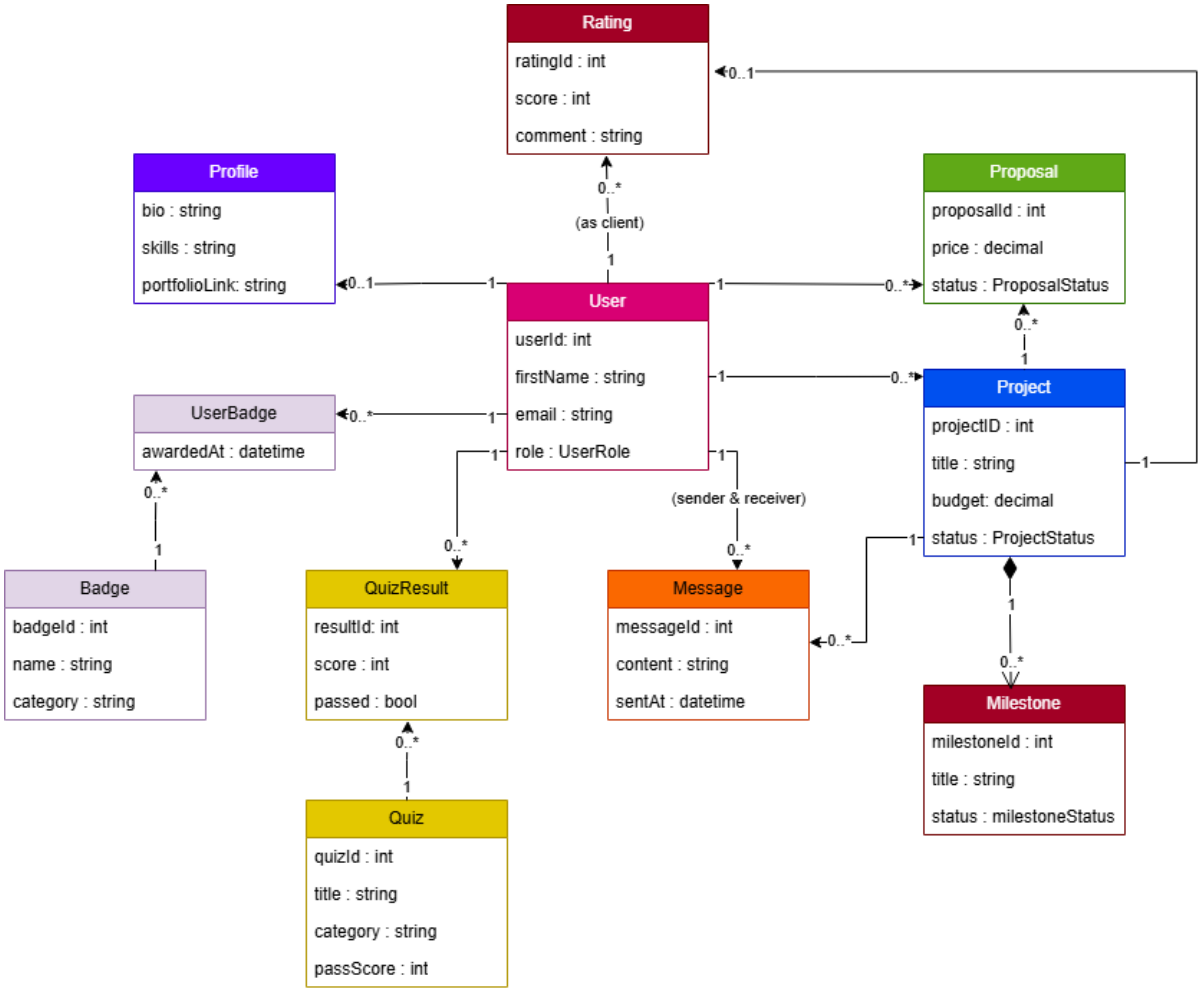


**Figure 8- Class Diagram**

The class diagram in Figure 8 gives a clear snapshot of how SharkTalent is structured internally. Instead of thinking in layers, the system is built around a set of core objects that represent the real things happening on the platform users, projects, proposals, milestones, quizzes, badges, messages, and ratings. Each class models something meaningful in the marketplace, and the relationships between them show how the system actually works.

At the centre of everything is the **User** class, which represents clients, freelancers, and admins. Extra profile details are kept in the **Profile** class, which is linked to each user on a one-to-one basis. Clients create **Projects**, and freelancers respond by sending **Proposals**. This naturally produces two simple relationships: one user can create many projects, and one freelancer can submit many proposals, but each proposal always belongs to exactly one user and one project.

Projects can be broken down into **Milestones**, which are tightly connected to the project (shown as a composition because milestones cannot exist on their own). Messaging and feedback also tie back into this structure: the **Message** class connects users who communicate about a project, while the **Rating** class captures the score and comment a client gives a freelancer once the work is completed. Both concepts link back to the user and project involved.

Skill verification is represented through the **Quiz**, **QuizQuestion**, **QuizResult**, and **Badge** classes. A quiz contains several questions, users can attempt quizzes multiple times, and those who pass earn badges. Because users can earn many badges and each badge can belong to many users the **UserBadge** class acts as the link between them.

Overall, the class diagram shows how the system's main features connect together in a clean and intuitive way. It highlights how responsibilities are separated, how objects depend on each other, and how SQLAlchemy can map these relationships directly into the database. This design forms the backbone of the system and guides how each feature is implemented in code.

### 4.3.2 Database Design

The database schema is structured into interlinked relational tables that support platform functionality. Each entity is normalized to reduce redundancy and maintain referential integrity.

The **Users** table serves as the foundation, linking to extended user details in **Profiles**, project ownership in **Projects**, proposal submissions in **Proposals**, and communication logs in **Messages**. Skill verification is implemented through **Quizzes**, **Quiz_Questions**, and **Quiz_Results**, with badges stored in **Badges** and assigned via **User_Badges**.

The **ER diagram** shown in Figure 9 visually represents these relationships.

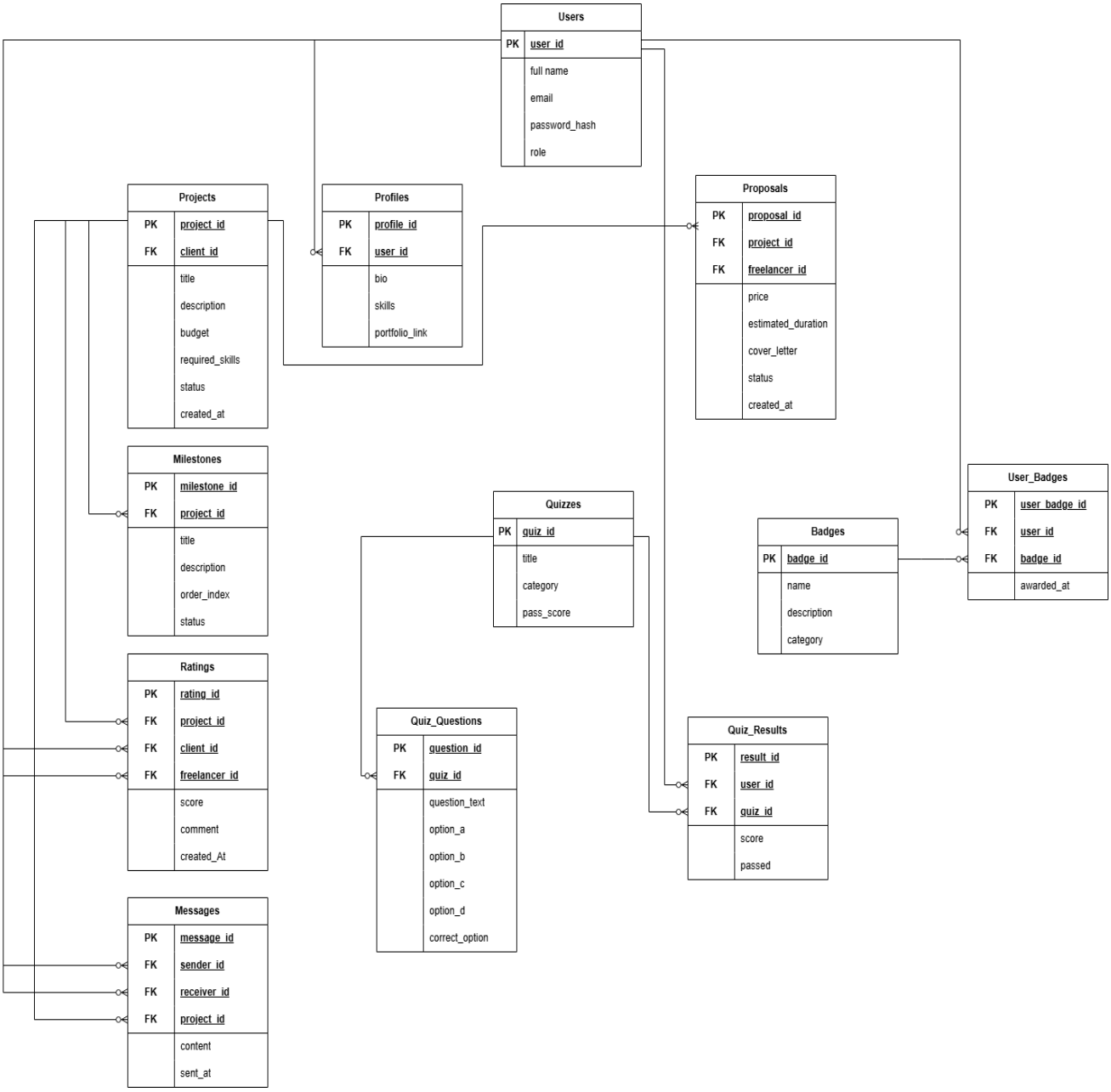
Users
PK user_id
full name
email
password_hash
role
Projects
PK project_id
FK client_id
title
description
budget
required_skills
status
created_at
Profiles
PK profile_id
FK user_id
bio
skills
portfolio_link
Proposals
PK proposal_id
FK project_id
FK freelancer_id
price
estimated_duration
cover_letter
status
created_at
Milestones
PK milestone_id
FK project_id
title
description
order_index
status
Quizzes
PK quiz_id
title
category
pass_score
Badges
PK badge_id
name
description
category
User_Badges
PK user_badge_id
FK user_id
FK badge_id
awarded_at
Ratings
PK rating_id
FK project_id
FK client_id
FK freelancer_id
score
comment
created_At
Quiz_Questions
PK question_id
FK quiz_id
question_text
option_a
option_b
option_c
option_d
correct_option
Quiz_Results
PK result_id
FK user_id
FK quiz_id
score
passed
Messages
PK message_id
FK sender_id
FK receiver_id
FK project_id
content
sent_at

**Figure 9- ERD**

Each table serves a clearly defined purpose:

- **Users:** Core authentication and role definition
- **Projects:** Client-generated work opportunities
- **Proposals:** Freelancer submissions constrained by platform rules
- **Milestones:** Project-specific task segmentation
- **Messages:** Client–freelancer communication
- **Badges and Quizzes:** Skill assessment system
- **Ratings:** Post-project evaluations

**ERD Explanation**

The Entity–Relationship Diagram (ERD) represents the logical data model of the SharkTalent platform and illustrates how the various components of the system interact through structured relational links. The model was designed using a normalized, modular approach to reduce redundancy, preserve data integrity, and support the platform's functional requirements such as project posting, proposal submission, messaging, skill verification, and milestone management. The following explanation outlines the purpose of each entity and the rationale behind the defined relationships.

**Users (Core Entity)**

The **Users** table is the central entity from which most relationships originate. It stores authentication and role information for all system participants, including clients, freelancers, and administrators. Each user has a unique identifier (user_id), and this primary key is referenced throughout the database to associate users with their related actions and records.
The Users entity forms the foundation for one-to-many relationships with Projects, Proposals, Quiz_Results, Ratings, Messages, and User_Badges.

**Profiles (Extended User Information)**

The **Profiles** table stores optional extended information such as bio, skills, and portfolio links. It is connected via a **one-to-one** relationship with Users through the foreign key user_id. This separation maintains normalization, ensuring that the core Users table remains lightweight while allowing richer profile data when available.
Like shown in figure below

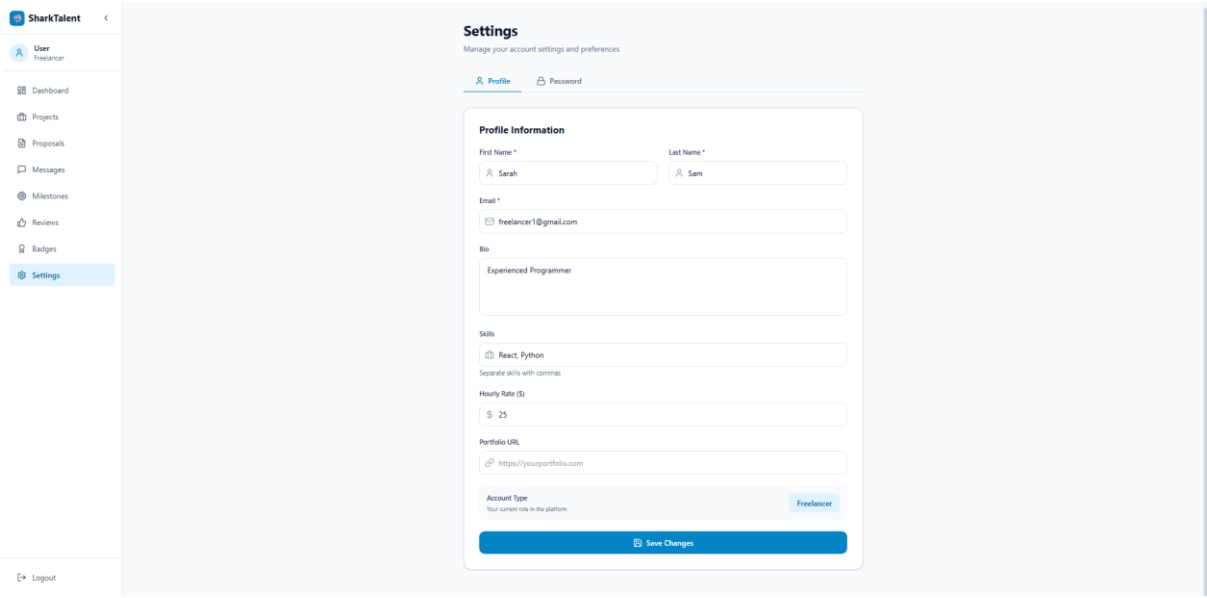


Figure 9.1- Profile information

## Projects (Client-Posted Work Opportunities)

Each project is created by a client (a user with the client role). This results in a **one-to-many** relationship between Users and Projects, where one user may own multiple projects but each project belongs to exactly one client.
Projects are further linked to dependent entities such as Milestones, Proposals, Messages, and Ratings. Like shown in figure below

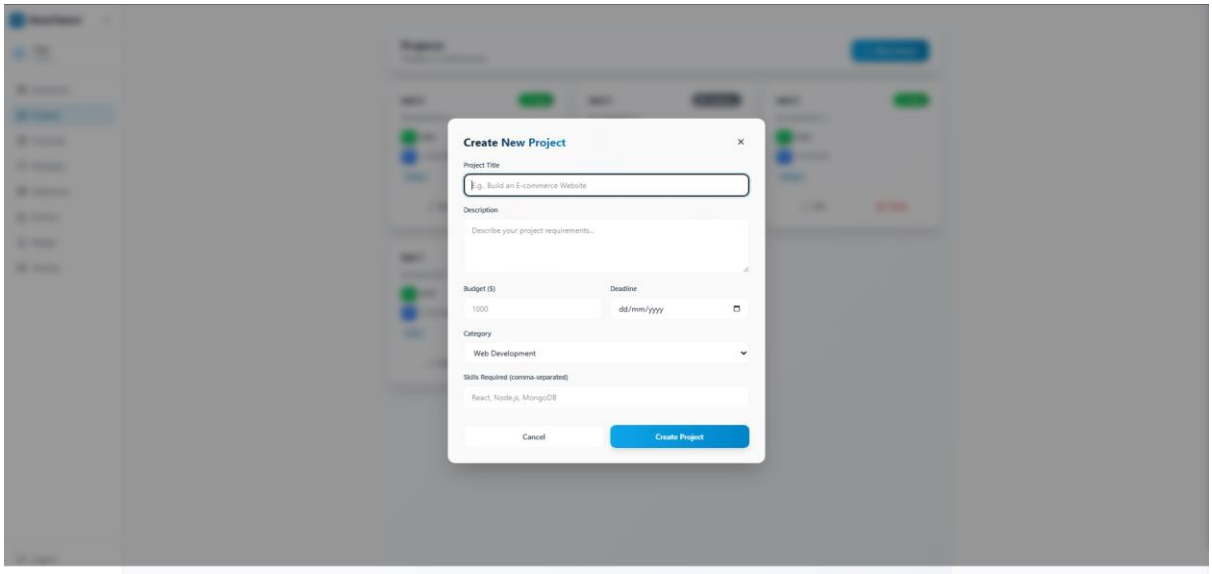


Figure 9.2- Projects to be posted by clients

### Proposals (Freelancer Applications to Projects)

The **Proposals** entity links freelancers to the projects they apply for.
Two relationships define its structure:

1. Users (as freelancers) → Proposals
2. Projects → Proposals

This creates a **many-to-one** relationship in both cases:

- A project can receive many proposals.
- A freelancer can submit many proposals.

The foreign keys freelancer_id and project_id enforce these links.
The ERD reflects this structure and supports the system's rule limiting freelancers to a maximum of three proposals per project, though this rule is enforced at the application logic level rather than at the database level. Like shown in figure below

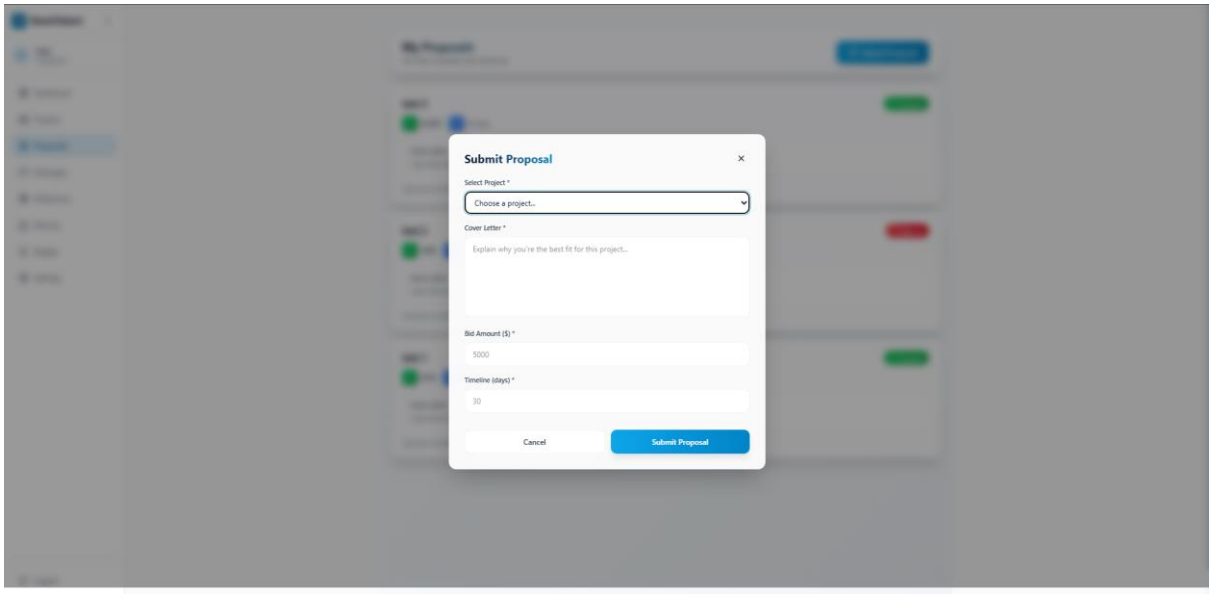


Figure 9.3- Freelancer submitting proposals

### Milestones (Structured Project Workflow)

Each project can be divided into multiple milestones, forming a **one-to-many** relationship between Projects and Milestones.
Milestones contain fields describing the task within the project and its current status (pending, in progress, submitted, approved).
This structure allows for controlled project progression and milestone tracking within the system. Like shown in figure below

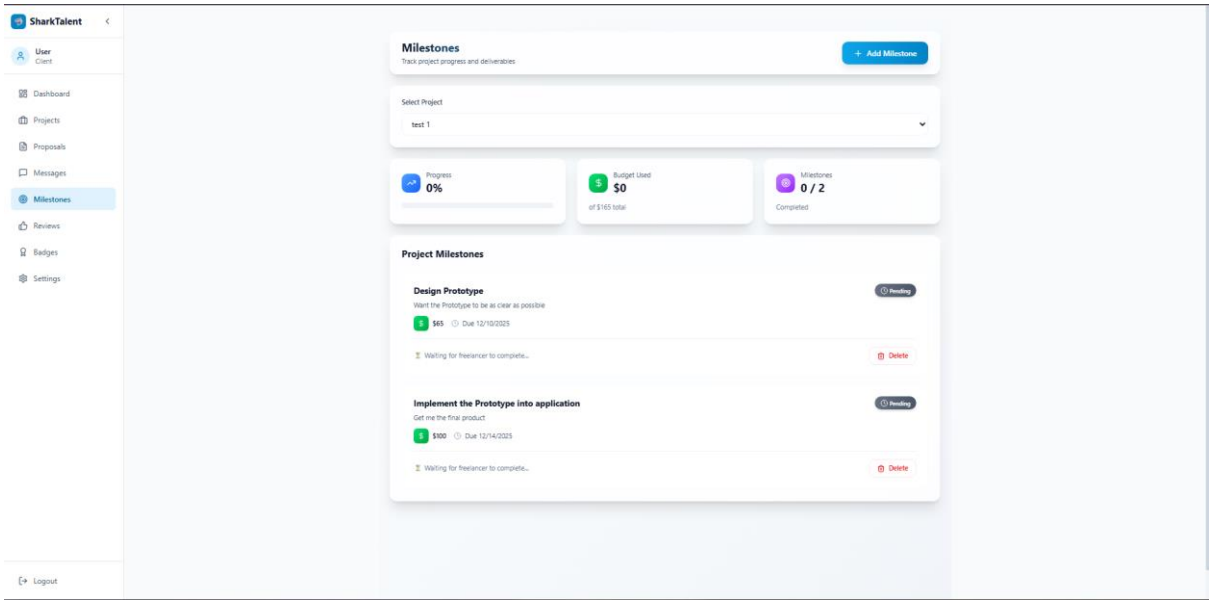


**Figure 9.4- milestone tracking**

### Ratings (Client Feedback for Completed Projects)

The **Ratings** table captures feedback submitted by clients regarding freelancers once a project is completed. This entity has **three foreign keys**, reflecting its multi-role connections:

- project_id links Ratings to the related project
- client_id links the rating to the user who submitted it
- freelancer_id links the rating to the user being evaluated

Thus, Ratings forms three **many-to-one** relationships:

- One project can have many ratings (though typically one per project)
- One client can give multiple ratings
- One freelancer can receive multiple ratings

This supports transparency and accountability across the platform. Like shown in figure below

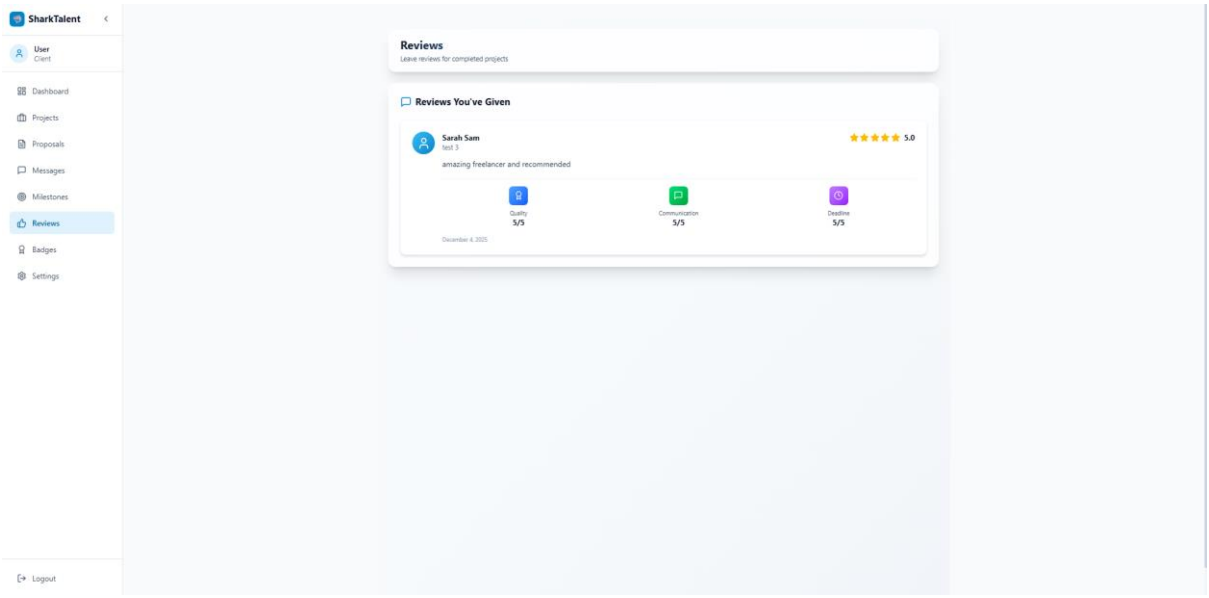


Figure 9.5- leaving reviews for the freelancer

### Messages (Client–Freelancer Communication)

The **Messages** entity allows structured communication within a project.
It includes **three foreign keys**:

- sender_id (FK → Users)
- receiver_id (FK → Users)
- project_id (FK → Projects)

This creates a set of relationships where:

- A user can send many messages
- A user can receive many messages
- A project can contain many messages

The dual FK relationship to Users (sender and receiver) is typical in messaging systems and accurately reflects communication flows. Like shown in figure below

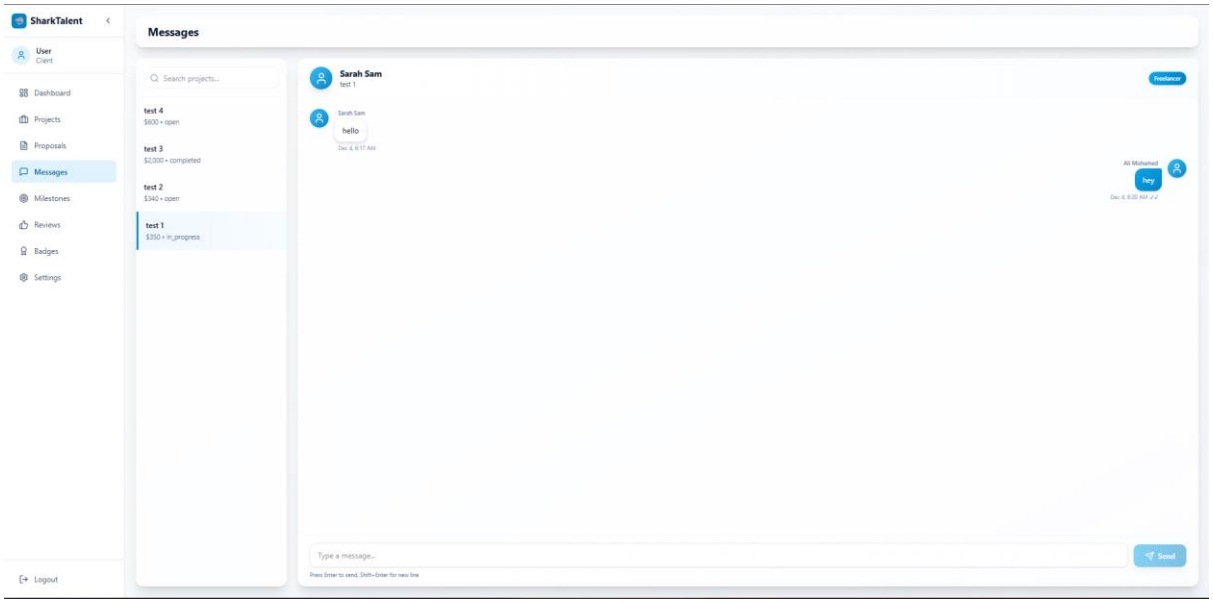


Figure 9.6- In app messaging feature

### Quizzes and Quiz_Questions (Skill Assessment Mechanism)

The assessment subsystem consists of two key entities:

1. **Quizzes** — defines the category and passing threshold
2. **Quiz_Questions** — questions that belong to each quiz

A **one-to-many** relationship exists between Quizzes and Quiz_Questions, using quiz_id as the foreign key.
This structure allows the system to manage dynamic assessments for skill verification.

### Quiz_Results (Tracking User Performance)

The **Quiz_Results** entity stores the outcome of each quiz attempt.
It holds foreign keys referencing both Users and Quizzes, forming two many-to-one relationships:

- One user may attempt many quizzes
- One quiz may generate many attempts

This structure supports badge assignment and provides evidence for the user's validated skills.

**Badges and User_Badges (Many-to-Many Skill Verification)**

The skill verification system uses a classic **many-to-many** relationship between Users and Badges.
This is implemented using the join table **User_Badges**, which contains foreign keys referencing both Users and Badges.
This design allows:

- Each user to earn multiple badges
- Each badge to be assigned to multiple users

Normalization through the join table ensures flexibility and scalability for the skill-verification system.

**Overall Structure and Rationale**

This ERD emphasizes:

- **Normalization** (removing duplicate data using join tables and separate entities)
- **Clear role separation** (Clients vs. Freelancers)
- **Traceability of actions** (messages, proposals, ratings all link back to users and projects)
- **Support for platform rules** (proposal limit, skill verification, milestone workflow)
- **Scalability** (future additions payment modules fit naturally)

The relationships expressed in the ERD ensure that the SharkTalent system is relationally sound, logically structured, and capable of supporting the platform's operational flow in a consistent and maintainable manner.

### 4.3.3 Sequence Milestone Design

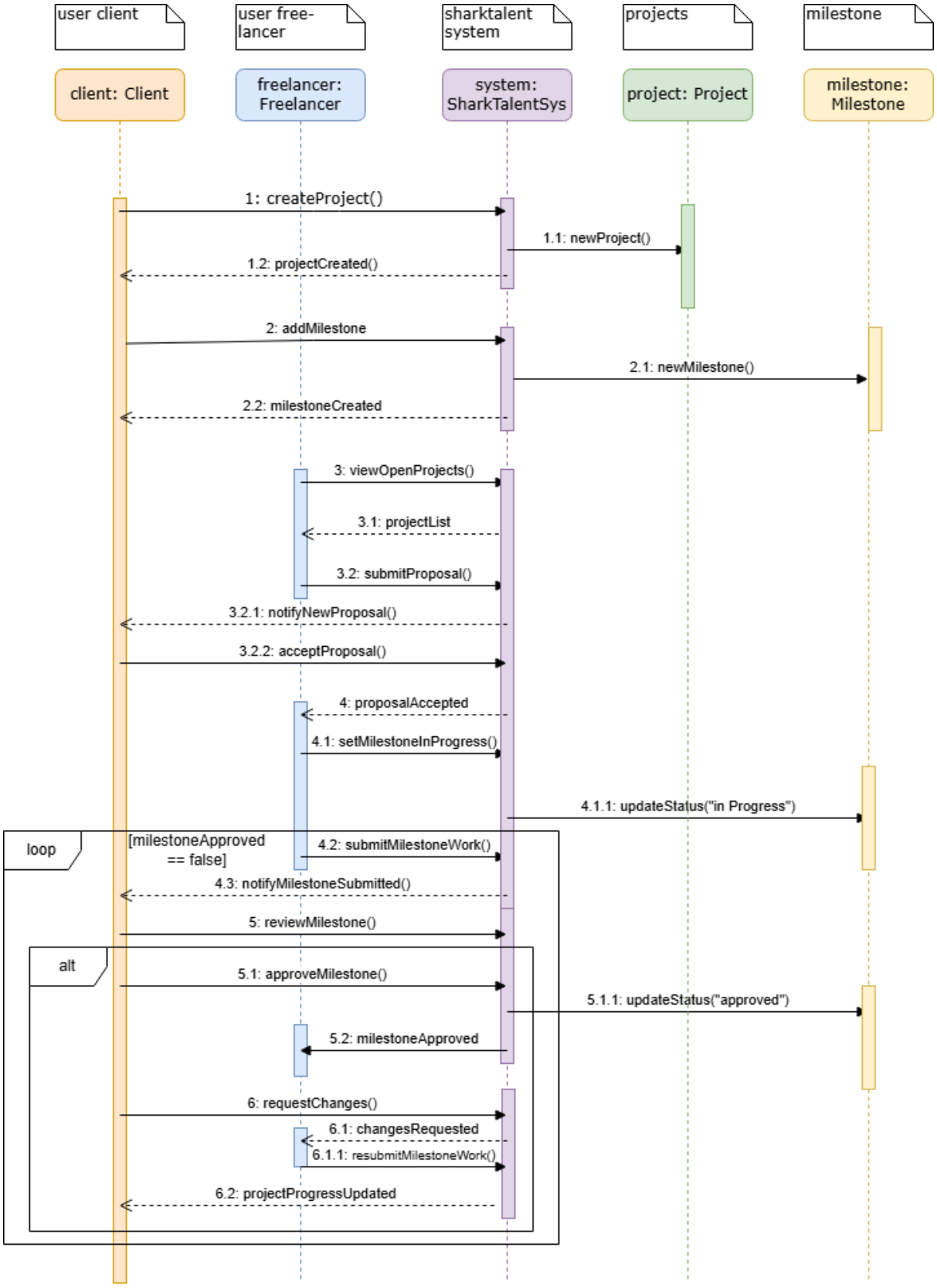


**Figure 10- Sequence Milestone Design**

- **Project and milestone creation (1, 1.1, 1.2, 2, 2.1, 2.2)**
  The client calls createProject(), the system creates a new Project object (newProject()) and confirms projectCreated(). The client then calls addMilestone(), the system creates a Milestone object (newMilestone()) and confirms milestoneCreated().
- **Freelancer discovers the project and applies (3, 3.1, 3.2, 3.2.1, 3.2.2, 4)**
  The freelancer requests viewOpenProjects(), the system returns a projectList, and the freelancer sends submitProposal(). The system notifies the client with notifyNewProposal(), the client answers with acceptProposal(), and the system informs the freelancer with proposalAccepted.
- **Milestone work is started (4.1, 4.1.1)**
  After being accepted, the freelancer calls setMilestoneInProgress(). The system updates the Milestone object status to **“in progress”** via updateStatus("in Progress").
- **Submission and review loop (loop [milestoneApproved == false], 4.2, 4.3, 5)**
  Inside the loop, the freelancer submits the finished work with submitMilestoneWork(), the system sends notifyMilestoneSubmitted() to the client, and the client triggers reviewMilestone().
- **Alt branch – approved vs changes requested (5.1–5.2, 6–6.1.1)**
  In the **approved** branch, the client calls approveMilestone(), the system sets the milestone status to **“approved”** (updateStatus("approved")) and notifies the freelancer via milestoneApproved. In the **changesRequested** branch, the client sends requestChanges(), the system notifies the freelancer with changesRequested, and the freelancer can respond with resubmitMilestoneWork(); the loop repeats until the approval condition is met.
- **Project progress update (6.2)**
  Once the milestone is finally approved and the loop ends, the system sends projectProgressUpdated to the client, reflecting the approved milestone in the overall project progress.

## 4.4 Implementation

The platform was implemented using industry-standard tools and technologies.

### 4.4.1 Frontend Implementation

The frontend's React components utilize modular state management and client-side routing. Axios is used for API communication, and conditional rendering supports role-specific interfaces. Form validations are enforced to reduce backend load and improve user experience.

### 4.4.2 Backend Implementation

Backend logic is divided into controllers, middleware, services, and database interaction modules. Middleware includes authentication guards and role-based access controls. Error handling follows Express best practices, ensuring clear feedback and secure execution.

### 4.4.3 Database Implementation

PostgreSQL schemas and constraints were established to enforce referential integrity. Indexes were added to frequently queried fields such as user IDs and project IDs to improve performance. Sequelize ORM or raw SQL queries (depending on implementation preference) manage database operations.

## 4.5 Coding Challenges and Common Pitfalls

During development, several technical challenges occurred:

- **Asynchronous React state updates** sometimes led to unexpected UI rendering.
- **Token expiration issues** resulted in repeated logouts during early testing.
- **SQL referential integrity violations** occurred due to incorrect foreign-key deletion rules.
- **Concurrent proposal submissions** exposed race conditions requiring additional validation.

These challenges highlight the importance of careful debugging, structured error handling, and database transaction discipline.

## 4.6 Testing Strategy

Testing followed the **category-partition model**, selected because the system includes multiple definable input categories such as user roles, project states, and evaluation outcomes. This model ensured comprehensive coverage of realistic user scenarios.

### 4.6.1 Test Categories

To construct a comprehensive set of test cases, the category-partition model was applied. The system requirements were analysed and decomposed into distinct input categories, each representing a dimension of system behaviour. These categories include:
**1** the user's role within the system (client, freelancer, or administrator),
**2** the validity of login credentials (valid credentials, invalid credentials, or missing fields),
**3** the current state of a project (open, in-progress, or completed),
**4** the number of proposals already submitted by a freelancer for a given project (0–2, the allowable range; exactly 3, the maximum limit; and more than 3, which should be rejected),
**5** the outcome of a quiz taken by a freelancer (pass or fail), and
**6** the stage of a project milestone within its state-machine workflow (pending, in progress, submitted, or approved).

These categories form the conceptual basis from which test combinations were derived, ensuring that both common and boundary conditions were systematically covered.

### 4.6.2 Functional Testing

Functional test cases validated critical system behaviors:

- Authentication (valid/invalid login)
- Project creation and validation
- Proposal submission with enforcement of the three-proposal rule
- Quiz scoring and badge assignment
- Messaging between users
- Proper restricted access based on user roles
- Milestone progression and approval

Each test case confirmed whether the system satisfied the functional requirements defined earlier.

#### 4.6.3 User Acceptance Testing (UAT)

User-acceptance testing was conducted with three participants one client and two freelancers who evaluated the platform's usability, operational clarity, and functional accuracy.

Feedback indicated:

- The interface was intuitive and accessible.
- The proposal limit system improved fairness.
- Skill-based badges were well-understood and perceived as useful.
- Messaging operated reliably, although users recommended additional interface enhancements.

## 4.7 Summary

This chapter provided a comprehensive account of the design methodology, architecture, implementation decisions, algorithmic innovations, and testing procedures utilized in the development of SharkTalent. Through a structured approach founded on layered architecture, modular decomposition, and category-partition testing, the platform successfully fulfils the requirements defined in previous chapters while introducing novel mechanisms for quality control and skill validation.

# CHAPTER 5 – RESULTS AND DISCUSSION

## 5.1 Findings

The development and evaluation of the SharkTalent platform produced a number of significant findings relating to usability, functionality, system behaviour, and user perceptions. The implemented prototype demonstrated that structured controls such as proposal limitations, skill-verification quizzes, milestone workflows, and built-in communication can meaningfully improve the quality and transparency of freelance interactions. These results support the overall concept: that a simplified, educationally oriented freelance platform can replicate essential marketplace operations without requiring complex AI-driven ranking systems.

A key finding was the behavioural impact of the proposal-limitation mechanism. Restricting freelancers to a maximum of three proposals per project reduced spam submissions and encouraged users to think more carefully about which opportunities to pursue. Similar observations have been reported in studies of online labour platforms, where unrestricted bidding leads to “race-to-the-bottom” behaviour and reduced proposal quality (Wood et al., 2019). In SharkTalent, testers reported that the limit pushed them to write more personalized proposals rather than sending identical pitches repeatedly.

Another important finding related to skill verification. The badge-based quiz system revealed that even lightweight assessments can significantly enhance user confidence. Test participants noted that badge visibility helped them understand whether a freelancer was genuinely qualified for a project, reducing the uncertainty typically associated with hiring unknown individuals online. This aligns with research suggesting that transparent verification systems reduce information asymmetry and increase trust in digital labour markets (Lehdonvirta et al., 2021).

The milestone workflow also proved effective. Users found that dividing projects into discrete tasks with clear approval points increased structure and accountability. Although the prototype does not integrate real payments, participants reported that the system felt realistic enough to reflect actual freelance work processes.

Some findings were unexpected. During early experimentation with messaging, real-time features using WebSockets were explored. Although not implemented in the final version due to scope constraints, this early exploration demonstrated that adding real-time interaction would be feasible in a future iteration. Another unintended discovery was the potential to generate analytics based on proposal performance, quiz attempts, and milestone efficiency features that were beyond the original scope but emerged naturally from the system’s data structure.

## 5.2 Goals Achieved

The majority of the project's objectives, as defined in Chapter 1, were fully or substantially achieved. The core aim—designing and implementing a controlled, structured freelance prototype—was successfully realized. The system incorporates the essential modules required for a functional freelance ecosystem, including:

- Role-based accounts
- Project creation and management
- Proposal submissions with enforced limits
- Messaging
- Milestone-based workflow
- Rating and feedback mechanisms

The proposal limitation rule was implemented exactly as specified and tested under multiple scenarios. It demonstrated consistent behaviour by rejecting fourth submissions and returning appropriate feedback messages. The objective of reducing spam was therefore met.

The quiz and badge system achieved its intended purpose of verifying freelancer competence through transparent, rule-based assessment. Quiz scoring, badge assignment, and badge-based filtering all operated reliably during testing. This supports existing academic findings that skill verification tools can improve perceived platform fairness (Sutherland & Jarrahi, 2018).

The milestone state machine worked as designed: transitions were enforced strictly, invalid transitions were blocked, and users were guided step-by-step. Although simplified, this mirrors real-world freelance workflows.

A few goals were only partially achieved. The platform does not yet support:

- Automated project recommendations
- Advanced profile customization
- Detailed administrative analytics

However, these limitations occurred primarily due to time constraints rather than conceptual failure. Importantly, documenting such limitations is academically valuable, as incomplete goals demonstrate critical reflection (Sharp et al., 2019).

## 5.3 Further Work

The development of SharkTalent revealed multiple areas where further enhancements would significantly increase platform capability, realism, and research potential.

Although the system design includes components for freelancer badges, skill-verification quizzes, and an administrative role, these features were not fully implemented due to the dynamic behaviour they require. Badge assignment depends on real-time quiz evaluation, score thresholds, and user-progress tracking, while the quiz module itself involves question management, scoring logic, and adaptive challenge levels. Similarly, the admin role would need advanced management tools, moderation capabilities, and live oversight mechanisms. These features were not omitted because of conceptual or technical limitations; rather, they were deferred to future development phases to maintain project scope, ensure system stability, and allow for a more rigorous and scalable implementation. Integrating these modules in future work would significantly enhance the platform by enabling automated skill verification, structured reputation building, and centralised administration.

### 5.3.1 Enhancing Skill Verification

Future work could strengthen the quiz system by introducing:

- Coding tasks evaluated automatically
- Adaptive difficulty assessments
- Timed examinations
- Integration with third-party testing (e.g., HackerRank)
- Badge levels (e.g., Bronze, Silver, Gold)

These enhancements would increase assessment reliability and better align the platform with real industry expectations.

### 5.3.2 Real-Time Features

Although the current system uses synchronous messaging, user feedback strongly indicated the desire for real-time communication. Incorporating WebSockets would enable:

- Live chat
- Instant milestone updates
- Real-time notifications

This would significantly improve interactivity and platform responsiveness.

### 5.3.3 Recommendation Systems

SharkTalent intentionally avoids opaque ranking algorithms, but future work could explore lightweight, transparent recommendation approaches, such as:

- Badge-based project recommendations

- Historical proposal success analytics
- Category matching based on previous work

These systems should maintain transparency to avoid algorithmic bias (O'Neil, 2016).

#### 5.3.4 Mobile Application

A mobile version built using React Native or Flutter would increase accessibility, making the platform suitable for practical use, not just academic demonstration.

#### 5.3.5 Administrative Tools

Future versions could include:

- Freelancer performance dashboards
- Detection flags for inactive or unreliable users
- Category-level analytics
- Project-completion statistics

These tools would support both research and operational scalability.

## 5.4 Ethical, Legal, and Social Considerations

Developing a platform that manages user-generated content, personal data, and behavioural controls introduces a range of ethical, legal, and societal implications.

**Ethical Concerns**

SharkTalent collects and stores personal information such as email addresses, quiz results, proposal content, and communication logs. Ethical system design requires ensuring that such data is used solely for legitimate operational purposes and protected against unauthorized access. The platform avoids opaque ranking or scoring algorithms, reducing the risk of discriminatory decision-making (Lepri et al., 2017).

**Legal Considerations**

If deployed beyond an academic environment, SharkTalent would need to comply with data-protection regulations such as GDPR. This includes user consent, data deletion rights, secure storage, and encryption. While the prototype does not involve real payments, future integration of financial transactions would require compliance with anti-fraud standards and payment regulations.

**Social Impact**

Freelance platforms significantly influence labour markets. Poorly designed systems can lead to exploitation, unfair competition, and opaque filtering. SharkTalent counters these issues by incorporating transparent rules, limiting proposal spam, and providing unbiased skill verification. These measures promote fairness, reduce psychological pressure on freelancers, and support a healthier digital labour environment.

# CHAPTER 6 – CONCLUSIONS

## Conclusion

The SharkTalent project set out to design and develop a structured, transparent, and academically oriented freelance platform, addressing long-standing issues in existing marketplaces such as spam, unreliable reviews, unverified skills, and fragmented communication. Through the implementation of a three-tier architecture, modular subsystems, and rule-based interaction design, the resulting prototype demonstrates that a simplified yet robust freelance environment can be achieved without complex ranking algorithms or monetary transactions.

The platform successfully provides core functionalities expected of a freelance system, including secure authentication, project posting, limited proposal submissions, competency-based skill verification, milestone tracking, and streamlined communication. These features collectively support the project's central goal: creating a safe, fair, and testable environment for clients and freelancers to interact.

A significant contribution of the project is the demonstration that behavioural constraints, such as the three-proposal limit, can improve application quality without requiring AI-driven sorting mechanisms. Likewise, the skill-badge system validates freelancer capabilities through transparent assessment rather than opaque ratings. These findings contribute not only to the platform's functionality but also to broader discussions regarding ethical and efficient freelance marketplace design.

Challenges encountered during implementation including asynchronous frontend behaviour, database constraints, and token management provided valuable learning experiences that informed the final design. Although some advanced features such as automated recommendations, badges and quizzes were not implemented due to time limitations, the platform remains extendable and technologically prepared for future enhancements.

Overall, SharkTalent represents a meaningful step toward rethinking freelance ecosystem models. It demonstrates that fairness, clarity, and structured workflows can coexist with usability and educational adaptability. Future iterations could further expand the platform's capabilities by developing the quiz and badge subsystems into fully automated assessment pipelines and integrating an administrative dashboard for enhanced system oversight. Introducing a dedicated admin role would allow for improved moderation, user management, dispute handling, and real-time platform monitoring features essential for scaling toward a production environment.

Additional development could also incorporate real-time communication via WebSockets, enabling instant messaging between clients and freelancers, improving responsiveness, and supporting a smoother milestone review cycle. Enhancing the proposal-review process for example through automated checks, recommendation engines, or spam detection could also increase platform quality and reduce friction for both parties. The envisioned badge and skill-verification subsystem offers

significant potential for future growth: adaptive quizzes, external skill integrations, and dynamic badge levels would create a measurable, transparent skill-ranking ecosystem that benefits freelancers seeking credibility and clients seeking reliability.

From an academic standpoint, the project also showcases the strengths of a domain-driven, class-diagram-based software architecture. The modelling process made it possible to clearly visualise system relationships, lifecycle constraints, and behavioural dependencies, helping ensure that the implemented components aligned with the platform's conceptual design. Even though some advanced modules were only partially implemented due to time constraints, their inclusion in the design demonstrates that the platform is extendable and structurally prepared for future enhancements.

In conclusion, SharkTalent highlights the value of thoughtful system design in solving real-world problems related to freelance work. It shows that even a student-built prototype can offer practical insights into marketplace fairness, structured collaboration, and skill verification laying a foundation for a more transparent and ethically aligned freelance ecosystem.

## References


Pressman, R. & Maxim, B., 2020. *Software Engineering: A Practitioner's Approach*. 9th ed. New York: McGraw-Hill.

Newman, S., 2019. *Building Microservices*. 2nd ed. Sebastopol: O'Reilly Media.

Sutherland, W. & Jarrahi, M.H., 2018. The sharing economy and digital platforms: A review and research agenda. *International Journal of Information Management*, 43, pp.328–341.

Wood, A., Graham, M., Lehdonvirta, V. & Hjorth, I., 2019. Good Gig, Bad Gig: Autonomy and Algorithmic Control in the Global Gig Economy. *Work, Employment and Society*, 33(1), pp.56–75.

Lehdonvirta, V., Kässi, O., Hjorth, I., Barnard, H. & Graham, M., 2021. Online Labour Markets and the Persistence of Personal Networks. *Journal of Management Studies*, 58(1), pp.53–81.

Lepri, B., Oliver, N., Letouzé, E., Pentland, A. & Vinck, P., 2017. Fair, transparent, and accountable algorithmic decision-making processes. *Philosophy & Technology*, 31, pp.611–627.

O'Neil, C., 2016. *Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy*. New York: Crown Publishing.

React Documentation, 2024. *React Official Documentation*. Available at: https://react.dev/ [Accessed 5 December 2025].

Node.js Foundation, 2024. *Node.js v20 Documentation*. Available at: https://nodejs.org/ [Accessed 5 December 2025].

Express.js, 2024. *Express Application Framework Documentation*. Available at: https://expressjs.com/ [Accessed 5 December 2025].

PostgreSQL Global Development Group, 2024. *PostgreSQL 16 Documentation*. Available at: https://www.postgresql.org/docs/ [Accessed 5 December 2025].

Upwork Inc., 2024. *Upwork Marketplace Overview*. Available at: https://www.upwork.com/ [Accessed 5 December 2025].

Fiverr International Ltd., 2024. *Fiverr Platform Structure and Guidelines*. Available at: https://www.fiverr.com/ [Accessed 5 December 2025].

Toptal LLC., 2024. *Toptal Talent Screening Process*. Available at: https://www.toptal.com/ [Accessed 5 December 2025].

Mozilla Developer Network (MDN), 2024. *WebSockets Overview*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API [Accessed 5 December 2025].

European Union, 2018. *General Data Protection Regulation (GDPR)*. Available at: https://gdpr.eu/ [Accessed 5 December 2025].

Amazon Web Services, 2024. *Serverless Computing Overview*. Available at: https://aws.amazon.com/serverless/ [Accessed 5 December 2025].